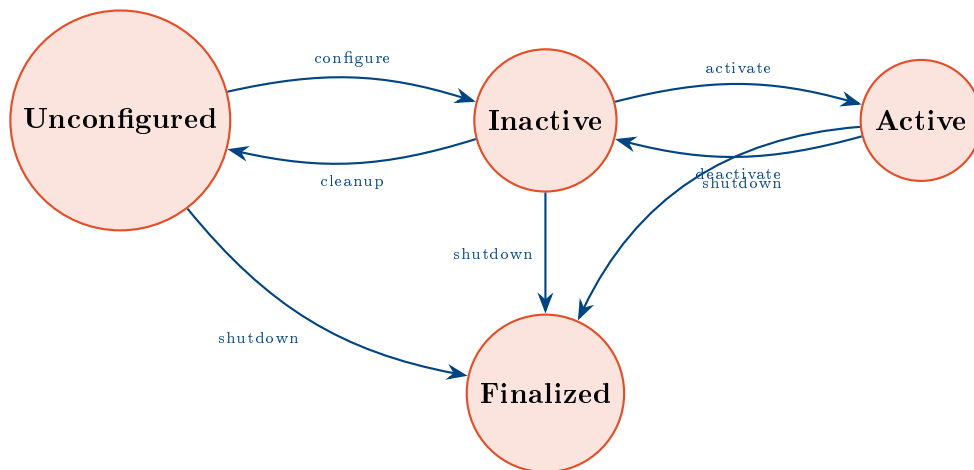


ROS 2 Lifecycle Nodes

A Complete Tutorial with Python & C++ Examples

Covers: Humble / Iron / Jazzy distributions



Version 1.0 • May 5, 2026

This document is a self-contained tutorial on ROS 2 *Lifecycle Nodes*. It includes full code examples in both Python and C++, build-system files, command-line usage, and a hands-on walkthrough. [Appendix A](#) covers installation of ROS 2 and workspace preparation on Ubuntu Linux.

Contents

1	Introduction	3
1.1	What is a Lifecycle Node?	3
1.2	Why Use Lifecycle Nodes?	3
1.3	Scope of this Tutorial	3
2	The Lifecycle State Machine	3
2.1	Primary States	3
2.2	Transition States	4
2.3	Transition Callbacks	4
2.4	Full State-Machine Diagram	4
3	Lifecycle Nodes in Python	5
3.1	Package Structure	5
3.2	package.xml	5
3.3	setup.py	6
3.4	Python – Lifecycle Talker (Publisher Node)	6
3.5	Python – Lifecycle Listener (Subscriber Node)	10
3.6	Python – Programmatic Lifecycle Manager	11
4	Lifecycle Nodes in C++	14
4.1	Package Structure	14
4.2	package.xml	14
4.3	CMakeLists.txt	15
4.4	C++ – Lifecycle Talker Header	16
4.5	C++ – Lifecycle Talker Implementation	17
4.6	C++ – Lifecycle Listener Header	20
4.7	C++ – Lifecycle Listener Implementation	21
4.8	C++ – Lifecycle Manager	22
5	Building the Packages	25
5.1	Workspace Layout	26
5.2	Building	26
5.3	Building Individual Packages	26
6	Running and Testing	26
6.1	Launching the Nodes	26
6.2	Manual Transitions via CLI	27
6.3	Using the Programmatic Manager	27
6.4	Monitoring State via Topic	27
7	Advanced Topics	28
7.1	Using Parameters Inside Lifecycle Nodes	28
7.2	Error Handling and Recovery	29
7.3	Using a Launch File with Lifecycle Nodes	30
7.4	Introspecting Lifecycle Services	31
8	Best Practices	31

9 Quick Reference Card	33
A Installing ROS 2 and Setting Up a Development Workspace on Ubuntu	33
A.1 Supported Ubuntu Versions	33
A.2 Step 1 – Set the Locale	33
A.3 Step 2 – Add the ROS 2 APT Repository	33
A.4 Step 3 – Install ROS 2	34
A.5 Step 4 – Initialise rosdep	34
A.6 Step 5 – Source the Installation	34
A.7 Step 6 – Create and Initialise a Colcon Workspace	35
A.8 Step 7 – Clone or Create Your Packages	35
A.9 Step 8 – Install Package Dependencies	35
A.10 Step 9 – Build and Test	35
A.11 Step 10 – Useful Development Tools	36
A.11.1 colcon Cheatsheet	36
A.11.2 ROS 2 Environment Utilities	36
A.11.3 Installing Extra Packages	36
A.11.4 Optional: Install Visual Studio Code with ROS Extension	37
A.12 Summary: Full Install Script	37
B Troubleshooting	38

1 Introduction

1.1 What is a Lifecycle Node?

In standard ROS 2, a node starts executing immediately upon creation, and the only way to stop it is to kill the process. This makes it difficult to coordinate the startup and shutdown of complex multi-node systems.

Lifecycle Nodes (also called *managed nodes*) address this by giving each node a well-defined state machine. A lifecycle node does not begin meaningful work until it is explicitly told to do so by an external manager or by user commands. The operator can also pause, reconfigure, and cleanly shut down the node at run-time without restarting the whole system.

Lifecycle nodes are standardised in [REP 2006](#) and implemented in the `rclcpp_lifecycle` (C++) and `rclpy` (Python, via `LifecycleNode`) packages.

1.2 Why Use Lifecycle Nodes?

- **Deterministic start-up:** ensure all dependencies are ready before a node becomes active.
- **Safe configuration:** allocate hardware resources only when needed; release them cleanly on shutdown.
- **Fault isolation:** deactivate a misbehaving node without restarting the entire robot pipeline.
- **System orchestration:** a *lifecycle manager* (e.g. from `nav2_lifecycle_manager`) can supervise multiple nodes as a unit.
- **Better testing:** each state transition can be unit-tested independently.

1.3 Scope of this Tutorial

1. The lifecycle state machine in detail.
2. Writing lifecycle nodes in **Python** and **C++**.
3. Building with `colcon` and `CMake`/`setuptools`.
4. Triggering transitions from the command line and programmatically.
5. Monitoring state via services and topics.
6. Implementing a simple lifecycle manager.
7. Best practices and common pitfalls.
8. Appendix: installing ROS 2 and setting up a workspace on Ubuntu.

2 The Lifecycle State Machine

2.1 Primary States

A lifecycle node always resides in one of the following *primary states*:

State	Description
Unconfigured	Node has been created but not yet configured. No publishers, subscribers, or timers are active.
Inactive	Node has been configured (resources allocated) but is not processing data.
Active	Node is fully operational and processing data.
Finalized	Node has been shut down and cannot be restarted.

2.2 Transition States

Between primary states the node briefly enters a *transition state* while executing callback logic. If the callback succeeds the node moves to the target primary state; if it fails the node moves to an error state instead.

Transition	From	To (success)	To (failure)
configure	Unconfigured	Inactive	Unconfigured
activate	Inactive	Active	Inactive
deactivate	Active	Inactive	Active
cleanup	Inactive	Unconfigured	Inactive
shutdown	Any primary	Finalized	Finalized
error	Any	Unconfigured	Finalized

2.3 Transition Callbacks

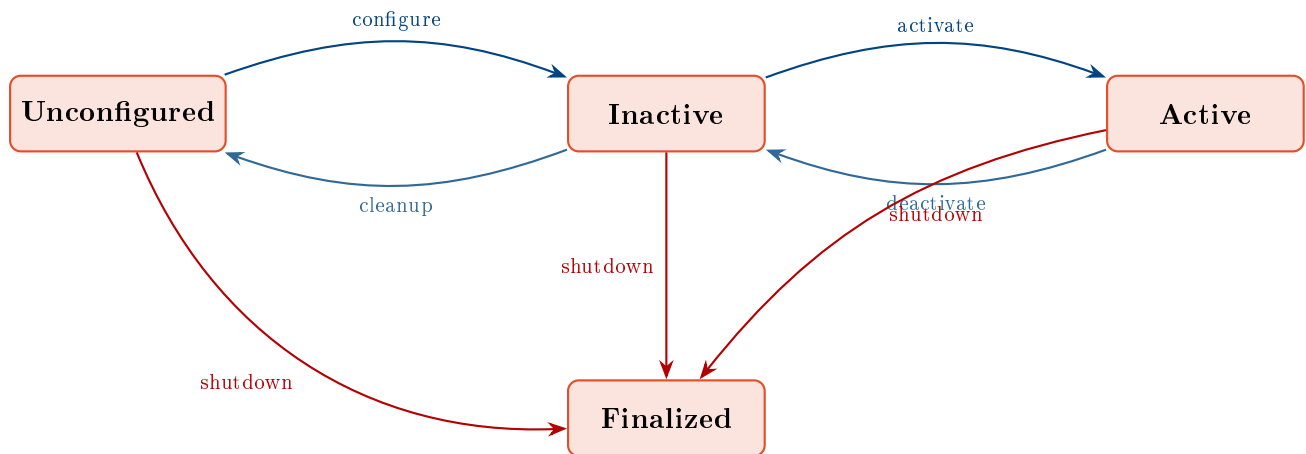
Each transition maps to a callback that you override in your node class:

Callback	Triggered by
on_configure	configure transition
on_activate	activate transition
on_deactivate	deactivate transition
on_cleanup	cleanup transition
on_shutdown	shutdown transition
on_error	Error recovery

All callbacks must return a *transition result*:

- **Python:** return `TransitionCallbackReturn.SUCCESS`, `.FAILURE`, or `.ERROR`.
- **C++:** return `CallbackReturn::SUCCESS`, `CallbackReturn::FAILURE`, or `CallbackReturn::ERROR`.

2.4 Full State-Machine Diagram



3 Lifecycle Nodes in Python

3.1 Package Structure

```

my_lifecycle_py/
  my_lifecycle_py/
    __init__.py
    lifecycle_talker.py      # publisher lifecycle node
    lifecycle_listener.py    # subscriber lifecycle node
    lifecycle_manager.py     # simple manager node
  resource/
    my_lifecycle_py
  package.xml
  setup.cfg
  setup.py

```

Listing 1: Python package layout

3.2 package.xml

```

1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.
  xsd"
3       schematypens="http://www.w3.org/2001/XMLSchema"?>
4 <package format="3">
5   <name>my_lifecycle_py</name>
6   <version>0.1.0</version>
7   <description>ROS 2 lifecycle node examples in Python</description>
8   <maintainer email="dev@example.com">Developer</maintainer>
9   <license>Apache-2.0</license>
10
11   <exec_depend>rclpy</exec_depend>
12   <exec_depend>lifecycle_msgs</exec_depend>
13   <exec_depend>std_msgs</exec_depend>
14
15   <test_depend>ament_copyright</test_depend>
16   <test_depend>ament_flake8</test_depend>

```

```

17 <test_depend>ament_pep257</test_depend>
18
19 <export>
20 <build_type>ament_python</build_type>
21 </export>
22 </package>

```

Listing 2: package.xml for the Python package

3.3 setup.py

```

1 from setuptools import setup
2
3 package_name = 'my_lifecycle_py'
4
5 setup(
6     name=package_name,
7     version='0.1.0',
8     packages=[package_name],
9     data_files=[
10         ('share/ament_index/resource_index/packages',
11          ['resource/' + package_name]),
12         ('share/' + package_name, ['package.xml']),
13     ],
14     install_requires=['setuptools'],
15     zip_safe=True,
16     maintainer='Developer',
17     maintainer_email='dev@example.com',
18     description='ROS 2 lifecycle node examples in Python',
19     license='Apache-2.0',
20     entry_points={
21         'console_scripts': [
22             'lifecycle_talker = my_lifecycle_py.lifecycle_talker:main',
23             'lifecycle_listener = my_lifecycle_py.lifecycle_listener:main',
24             'lifecycle_manager = my_lifecycle_py.lifecycle_manager:main',
25         ],
26     },
27 )

```

Listing 3: setup.py

3.4 Python – Lifecycle Talker (Publisher Node)

The following example implements a lifecycle node that publishes a `std_msgs/String` message only while in the *Active* state. It demonstrates the four main transition callbacks.

```

1 # Copyright 2024 Developer
2 # SPDX-License-Identifier: Apache-2.0
3 """Lifecycle talker: publishes only while Active."""

```

```

4
5 import rclpy
6 from rclpy.lifecycle import LifecycleNode
7 from rclpy.lifecycle import TransitionCallbackReturn
8 from rclpy.lifecycle import State
9 from rclpy.timer import Timer
10 from std_msgs.msg import String
11
12
13 class LifecycleTalker(LifecycleNode):
14     """A lifecycle node that publishes 'Hello' messages."""
15
16     def __init__(self, node_name: str, **kwargs) -> None:
17         """Initialise without creating ROS entities yet."""
18         super().__init__(node_name, **kwargs)
19         self._pub = None
20         self._timer: Timer | None = None
21         self._count: int = 0
22         self.get_logger().info('LifecycleTalker created (
                Unconfigured)')
23
24     # -- Transition callbacks
25     -----
26
27     def on_configure(self, state: State) ->
28     TransitionCallbackReturn:
29         """Allocate resources: create publisher.
30
31         Called by the 'configure' transition.
32         Returning SUCCESS moves the node to Inactive.
33         """
34
35         self.get_logger().info(
36             f'Configuring from state: {state.label}')
37
38         # Create publisher -- it exists but is NOT active yet
39         self._pub = self.create_lifecycle_publisher(
40             String, 'lifecycle_chatter', 10)
41
42         self._count = 0
43         self.get_logger().info('Publisher created.')
44         return TransitionCallbackReturn.SUCCESS
45
46     def on_activate(self, state: State) -> TransitionCallbackReturn
47     :
48         """Start publishing: create a timer that fires every second
49         .
50
51         Called by the 'activate' transition.
52         Returning SUCCESS moves the node to Active.
53         """
54
55         self.get_logger().info(

```



```

50         f'Activating from state: {state.label}'))
51
52     # Timer is created here so it fires only in Active state
53     self._timer = self.create_timer(1.0, self._timer_callback)
54
55     # IMPORTANT: call super().on_activate() to activate the
56     # publisher
57     return super().on_activate(state)
58
59 def on_deactivate(self, state: State) ->
60     TransitionCallbackReturn:
61     """Pause publishing: destroy the timer.
62
63     Called by the 'deactivate' transition.
64     Returning SUCCESS moves the node to Inactive.
65     """
66
67     self.get_logger().info(
68         f'Deactivating from state: {state.label}'))
69
70     if self._timer:
71         self._timer.destroy()
72         self._timer = None
73
74     # Call super to deactivate the publisher
75     return super().on_deactivate(state)
76
77 def on_cleanup(self, state: State) -> TransitionCallbackReturn:
78     """Release all resources.
79
80     Called by the 'cleanup' transition.
81     Returning SUCCESS moves the node back to Unconfigured.
82     """
83
84     self.get_logger().info(
85         f'Cleaning up from state: {state.label}'))
86
87     if self._timer:
88         self._timer.destroy()
89         self._timer = None
90
91     if self._pub:
92         self.destroy_publisher(self._pub)
93         self._pub = None
94
95     self._count = 0
96     self.get_logger().info('Resources released.')
97     return TransitionCallbackReturn.SUCCESS
98
99 def on_shutdown(self, state: State) -> TransitionCallbackReturn:
100     """Graceful shutdown -- called from any primary state."""
101     self.get_logger().info(

```

```

98         f'Shutting down from state: {state.label}'))
99
100     # Re-use cleanup logic
101     if self._timer:
102         self._timer.destroy()
103         self._timer = None
104     if self._pub:
105         self.destroy_publisher(self._pub)
106         self._pub = None
107
108     return TransitionCallbackReturn.SUCCESS
109
110 def on_error(self, state: State) -> TransitionCallbackReturn:
111     """Error recovery handler."""
112     self.get_logger().error(
113         f'Error in state: {state.label}. Attempting recovery.')
114     # Attempt to clean up; returning FAILURE sends to Finalized
115     return TransitionCallbackReturn.SUCCESS
116
117 # -- Internal
118 -----
119
120 def _timer_callback(self) -> None:
121     """Publish a message. Only called while Active."""
122     if self._pub and self._pub.is_activated:
123         msg = String()
124         msg.data = f'Hello, lifecycle world! count={self._count}'
125         self._pub.publish(msg)
126         self.get_logger().info(f'Published: "{msg.data}"')
127         self._count += 1
128
129 def main(args=None):
130     rclpy.init(args=args)
131     node = LifecycleTalker('lifecycle_talker')
132     executor = rclpy.executors.SingleThreadedExecutor()
133     executor.add_node(node)
134     try:
135         executor.spin()
136     except KeyboardInterrupt:
137         pass
138     finally:
139         node.destroy_node()
140         rclpy.shutdown()
141
142
143 if __name__ == '__main__':
144     main()

```

Listing 4: lifecycle_talker.py – full example

Note

Key point: use `create_lifecycle_publisher()` instead of `create_publisher()`. A lifecycle publisher is automatically *activated* and *deactivated* with the node, preventing messages from being published in the wrong state. Always call `super().on_activate(state)` and `super().on_deactivate(state)` to trigger this behaviour.

3.5 Python – Lifecycle Listener (Subscriber Node)

```

1  # Copyright 2024 Developer
2  # SPDX-License-Identifier: Apache-2.0
3  """Lifecycle listener: receives messages only while Active."""
4
5  import rclpy
6  from rclpy.lifecycle import LifecycleNode
7  from rclpy.lifecycle import TransitionCallbackReturn
8  from rclpy.lifecycle import State
9  from std_msgs.msg import String
10
11
12  class LifecycleListener(LifecycleNode):
13      """A lifecycle node that subscribes to the chatter topic."""
14
15      def __init__(self, node_name: str, **kwargs) -> None:
16          super().__init__(node_name, **kwargs)
17          self._sub = None
18          self.get_logger().info('LifecycleListener created (
19                                  Unconfigured)')
20
21      def on_configure(self, state: State) ->
22          TransitionCallbackReturn:
23          self.get_logger().info('Configuring listener...')
24          # Subscription is created here; messages are queued but the
25          # callback fires only when the node is Active.
26          self._sub = self.create_subscription(
27              String,
28              'lifecycle_chatter',
29              self._chatter_callback,
30              10,
31          )
32          return TransitionCallbackReturn.SUCCESS
33
34      def on_activate(self, state: State) -> TransitionCallbackReturn:
35          self.get_logger().info('Activating listener...')
36          return super().on_activate(state)
37
38      def on_deactivate(self, state: State) ->
39          TransitionCallbackReturn:
40          self.get_logger().info('Deactivating listener...')
41          return super().on_deactivate(state)

```

```

39
40     def on_cleanup(self, state: State) -> TransitionCallbackReturn:
41         self.get_logger().info('Cleaning up listener...')
42         if self._sub:
43             self.destroy_subscription(self._sub)
44             self._sub = None
45         return TransitionCallbackReturn.SUCCESS
46
47     def on_shutdown(self, state: State) -> TransitionCallbackReturn:
48         self.get_logger().info('Shutting down listener...')
49         if self._sub:
50             self.destroy_subscription(self._sub)
51             self._sub = None
52         return TransitionCallbackReturn.SUCCESS
53
54     def _chatter_callback(self, msg: String) -> None:
55         self.get_logger().info(f'Received: "{msg.data}"')
56
57
58 def main(args=None):
59     rclpy.init(args=args)
60     node = LifecycleListener('lifecycle_listener')
61     executor = rclpy.executors.SingleThreadedExecutor()
62     executor.add_node(node)
63     try:
64         executor.spin()
65     except KeyboardInterrupt:
66         pass
67     finally:
68         node.destroy_node()
69         rclpy.shutdown()
70
71
72 if __name__ == '__main__':
73     main()

```

Listing 5: lifecycle_listener.py

3.6 Python – Programmatic Lifecycle Manager

The following node drives another lifecycle node through its states programmatically by calling lifecycle change-state services.

```

1  # Copyright 2024 Developer
2  # SPDX-License-Identifier: Apache-2.0
3  """Simple lifecycle manager that drives another node through its
   states."""
4
5  import time
6  import rclpy
7  from rclpy.node import Node

```

```

8 from lifecycle_msgs.srv import ChangeState, GetState
9 from lifecycle_msgs.msg import Transition
10
11
12 TRANSITIONS = {
13     'configure': Transition.TRANSITION_CONFIGURE,
14     'activate': Transition.TRANSITION_ACTIVATE,
15     'deactivate': Transition.TRANSITION_DEACTIVATE,
16     'cleanup': Transition.TRANSITION_CLEANUP,
17     'shutdown': Transition.TRANSITION_UNCONFIGURED_SHUTDOWN,
18 }
19
20
21 class LifecycleManager(Node):
22     """Drives a target lifecycle node through a predefined sequence
23     ."""
24
25     def __init__(self, target_node: str = 'lifecycle_talker') ->
26         None:
27         super().__init__('lifecycle_manager')
28         self._target = target_node
29
30         # Create service clients
31         self._change_state = self.create_client(
32             ChangeState,
33             f'/{target_node}/change_state',
34         )
35         self._get_state = self.create_client(
36             GetState,
37             f'/{target_node}/get_state',
38         )
39         self.get_logger().info(
40             f'LifecycleManager ready to manage: {target_node}')
41
42         # -- Public API
43         -----
44
45     def get_state(self) -> str | None:
46         """Return the current primary state label of the managed
47         node."""
48         if not self._get_state.wait_for_service(timeout_sec=5.0):
49             self.get_logger().error('get_state service not
50             available')
51             return None
52         future = self._get_state.call_async(GetState.Request())
53         rclpy.spin_until_future_complete(self, future, timeout_sec
54             =5.0)
55         if future.result():
56             return future.result().current_state.label
57         return None

```

```

53     def change_state(self, transition_name: str) -> bool:
54         """Trigger a named transition. Returns True on success."""
55         if transition_name not in TRANSITIONS:
56             self.get_logger().error(
57                 f'Unknown transition: {transition_name}')
58             return False
59
60         if not self._change_state.wait_for_service(timeout_sec=5.0):
61             self.get_logger().error('change_state service not
62                                     available')
63             return False
64
65         req = ChangeState.Request()
66         req.transition.id = TRANSITIONS[transition_name]
67         req.transition.label = transition_name
68
69         future = self._change_state.call_async(req)
70         rclpy.spin_until_future_complete(self, future, timeout_sec
71                                         =10.0)
72
73         if future.result() and future.result().success:
74             state = self.get_state()
75             self.get_logger().info(
76                 f'Transition "{transition_name}" succeeded. ',
77                 f'New state: {state}')
78             return True
79         else:
80             self.get_logger().error(
81                 f'Transition "{transition_name}" FAILED.')
82             return False
83
84     def run_demo_sequence(self) -> None:
85         """Run configure -> activate -> (wait) -> deactivate ->
86             cleanup."""
87         steps = [
88             ('configure', 2.0),
89             ('activate', 5.0),
90             ('deactivate', 1.0),
91             ('cleanup', 1.0),
92         ]
93         for transition, wait in steps:
94             if not self.change_state(transition):
95                 self.get_logger().error(
96                     f'Aborting sequence at: {transition}')
97                 return
98             time.sleep(wait)
99
100         self.get_logger().info('Demo sequence complete.')

```

```

100 def main(args=None):
101     rclpy.init(args=args)
102     manager = LifecycleManager(target_node='lifecycle_talker')
103
104     # Wait briefly for the talker to come up
105     time.sleep(2.0)
106     manager.run_demo_sequence()
107
108     manager.destroy_node()
109     rclpy.shutdown()
110
111
112 if __name__ == '__main__':
113     main()

```

Listing 6: lifecycle_manager.py

4 Lifecycle Nodes in C++

4.1 Package Structure

```

my_lifecycle_cpp/
include/
  my_lifecycle_cpp/
    lifecycle_talker.hpp
    lifecycle_listener.hpp
    lifecycle_manager.hpp
src/
  lifecycle_talker.cpp
  lifecycle_listener.cpp
  lifecycle_manager.cpp
package.xml
CMakeLists.txt

```

Listing 7: C++ package layout

4.2 package.xml

```

1 <?xml version="1.0"?>
2 <package format="3">
3   <name>my_lifecycle_cpp</name>
4   <version>0.1.0</version>
5   <description>ROS 2 lifecycle node examples in C++</description>
6   <maintainer email="dev@example.com">Developer</maintainer>
7   <license>Apache-2.0</license>
8
9   <buildtool_depend>ament_cmake</buildtool_depend>
10
11   <depend>rclcpp</depend>
12   <depend>rclcpp_lifecycle</depend>
13   <depend>lifecycle_msgs</depend>

```

```

14   <depend>std_msgs</depend>
15
16   <test_depend>ament_lint_auto</test_depend>
17   <test_depend>ament_lint_common</test_depend>
18
19   <export>
20     <build_type>ament_cmake</build_type>
21   </export>
22 </package>

```

Listing 8: package.xml for the C++ package

4.3 CMakeLists.txt

```

cmake_minimum_required(VERSION 3.14)
project(my_lifecycle_cpp)

# C++17 required for std::optional, structured bindings, etc.
if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 17)
endif()
if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# Find dependencies
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_lifecycle REQUIRED)
find_package(lifecycle_msgs REQUIRED)
find_package(std_msgs REQUIRED)

set(DEPS
  rclcpp rclcpp_lifecycle lifecycle_msgs std_msgs)

# -- Lifecycle Talker
-----
add_executable(lifecycle_talker src/lifecycle_talker.cpp)
ament_target_dependencies(lifecycle_talker ${DEPS})
target_include_directories(lifecycle_talker PUBLIC
  ${<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>})

# -- Lifecycle Listener
-----
add_executable(lifecycle_listener src/lifecycle_listener.cpp)
ament_target_dependencies(lifecycle_listener ${DEPS})
target_include_directories(lifecycle_listener PUBLIC
  ${<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>})

# -- Lifecycle Manager
-----

```



```

add_executable(lifecycle_manager src/lifecycle_manager.cpp)
ament_target_dependencies(lifecycle_manager ${DEPS})
target_include_directories(lifecycle_manager PUBLIC
    $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>)

# Install
install(TARGETS
    lifecycle_talker lifecycle_listener lifecycle_manager
    DESTINATION lib/${PROJECT_NAME})

if(BUILD_TESTING)
    find_package(ament_lint_auto REQUIRED)
    ament_lint_auto_find_test_dependencies()
endif()

ament_package()

```

Listing 9: CMakeLists.txt

4.4 C++ – Lifecycle Talker Header

```

1 // Copyright 2024 Developer
2 // SPDX-License-Identifier: Apache-2.0
3 #ifndef MY_LIFECYCLE_CPP__LIFECYCLE_TALKER_HPP_
4 #define MY_LIFECYCLE_CPP__LIFECYCLE_TALKER_HPP_
5
6 #include <memory>
7 #include <string>
8
9 #include "rclcpp/rclcpp.hpp"
10 #include "rclcpp_lifecycle/lifecycle_node.hpp"
11 #include "rclcpp_lifecycle/lifecycle_publisher.hpp"
12 #include "std_msgs/msg/string.hpp"
13
14 namespace my_lifecycle_cpp
15 {
16
17 using CallbackReturn =
18     rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
19         CallbackReturn;
20
21 /**
22  * @brief Lifecycle talker node.
23  *
24  * Publishes std_msgs::msg::String messages at 1 Hz only while
25  * Active.
26  */
27 class LifecycleTalker : public rclcpp_lifecycle::LifecycleNode
28 {
29 public:
30     /**
31      * @brief Construct the talker.

```

```

30  * @param options Node options forwarded to LifecycleNode.
31  */
32  explicit LifecycleTalker(
33      const rclcpp::NodeOptions & options = rclcpp::NodeOptions());
34
35  ~LifecycleTalker() override = default;
36
37  // -- Transition callbacks
38  -----
39  CallbackReturn on_configure(const rclcpp_lifecycle::State & state
40      ) override;
41  CallbackReturn on_activate (const rclcpp_lifecycle::State &
42      state) override;
43  CallbackReturn on_deactivate(const rclcpp_lifecycle::State &
44      state) override;
45  CallbackReturn on_cleanup (const rclcpp_lifecycle::State &
46      state) override;
47  CallbackReturn on_shutdown (const rclcpp_lifecycle::State &
48      state) override;
49
50 private:
51  void timer_callback();
52
53  rclcpp_lifecycle::LifecyclePublisher<std_msgs::msg::String>::
54      SharedPtr pub_;
55  rclcpp::TimerBase::SharedPtr timer_;
56  size_t count_{0};
57 };
58
59 } // namespace my_lifecycle_cpp
60 #endif // MY_LIFECYCLE_CPP_LIFECYCLE_TALKER_HPP_

```

Listing 10: include/my_lifecycle_cpp/lifecycle_talker.hpp

4.5 C++ – Lifecycle Talker Implementation

```

1  // Copyright 2024 Developer
2  // SPDX-License-Identifier: Apache-2.0
3  #include "my_lifecycle_cpp/lifecycle_talker.hpp"
4
5  #include <chrono>
6  #include <memory>
7  #include <string>
8
9  using namespace std::chrono_literals;
10
11 namespace my_lifecycle_cpp
12 {
13
14  LifecycleTalker::LifecycleTalker(const rclcpp::NodeOptions &
15      options)
16  : rclcpp_lifecycle::LifecycleNode("lifecycle_talker", options)

```

```

16 {
17     RCLCPP_INFO(get_logger(), "LifecycleTalker created (Unconfigured)");
18 }
19
20 // -- on_configure
21 -----
22 CallbackReturn
23 LifecycleTalker::on_configure(const rclcpp_lifecycle::State & state)
24 {
25     RCLCPP_INFO(get_logger(),
26         "Configuring from state: %s", state.label().c_str());
27
28     // Create a lifecycle publisher.
29     // It is inactive until on_activate() is called.
30     pub_ = create_lifecycle_publisher<std_msgs::msg::String>(
31         "lifecycle_chatter", 10);
32
33     count_ = 0;
34     RCLCPP_INFO(get_logger(), "Publisher created.");
35     return CallbackReturn::SUCCESS;
36 }
37
38 // -- on_activate
39 -----
40 CallbackReturn
41 LifecycleTalker::on_activate(const rclcpp_lifecycle::State & state)
42 {
43     RCLCPP_INFO(get_logger(),
44         "Activating from state: %s", state.label().c_str());
45
46     // Create the timer that triggers publishing
47     timer_ = create_wall_timer(
48         1s, std::bind(&LifecycleTalker::timer_callback, this));
49
50     // IMPORTANT: call the parent implementation to activate the
51     // publisher
52     LifecycleNode::on_activate(state);
53     return CallbackReturn::SUCCESS;
54 }
55
56 // -- on_deactivate
57 -----
58 CallbackReturn
59 LifecycleTalker::on_deactivate(const rclcpp_lifecycle::State &
60     state)
61 {
62     RCLCPP_INFO(get_logger(),
63         "Deactivating from state: %s", state.label().c_str());
64 }

```

```

60 // Destroy the timer to stop publishing
61 timer_.reset();
62
63 // Deactivate the publisher through the parent
64 LifecycleNode::on_deactivate(state);
65 return CallbackReturn::SUCCESS;
66 }
67
68 // -- on_cleanup
69 -----
69 CallbackReturn
70 LifecycleTalker::on_cleanup(const rclcpp_lifecycle::State & state)
71 {
72     RCLCPP_INFO(get_logger(),
73         "Cleaning up from state: %s", state.label().c_str());
74
75     timer_.reset();
76     pub_.reset();
77     count_ = 0;
78     RCLCPP_INFO(get_logger(), "Resources released.");
79     return CallbackReturn::SUCCESS;
80 }
81
82 // -- on_shutdown
83 -----
83 CallbackReturn
84 LifecycleTalker::on_shutdown(const rclcpp_lifecycle::State & state)
85 {
86     RCLCPP_INFO(get_logger(),
87         "Shutting down from state: %s", state.label().c_str());
88
89     timer_.reset();
90     pub_.reset();
91     return CallbackReturn::SUCCESS;
92 }
93
94 // -- timer_callback
95 -----
95 void LifecycleTalker::timer_callback()
96 {
97     if (!pub_ || !pub_->is_activated()) {
98         return;
99     }
100     auto msg = std_msgs::msg::String();
101     msg.data = "Hello, lifecycle world! count=" + std::to_string(
102         count_++);
103     pub_->publish(msg);
104     RCLCPP_INFO(get_logger(), "Published: '%s'", msg.data.c_str());
105 }
106 } // namespace my_lifecycle_cpp

```

```

107
108 // -- main
109
110 -----
111
109 int main(int argc, char ** argv)
110 {
111     rclcpp::init(argc, argv);
112     auto node = std::make_shared<my_lifecycle_cpp::LifecycleTalker>()
113         ;
114     rclcpp::spin(node->get_node_base_interface());
115     rclcpp::shutdown();
116     return 0;
117 }

```

Listing 11: src/lifecycle_talker.cpp

4.6 C++ – Lifecycle Listener Header

```

1 // Copyright 2024 Developer
2 // SPDX-License-Identifier: Apache-2.0
3 #ifndef MY_LIFECYCLE_CPP__LIFECYCLE_LISTENER_HPP_
4 #define MY_LIFECYCLE_CPP__LIFECYCLE_LISTENER_HPP_
5
6 #include <memory>
7
8 #include "rclcpp/rclcpp.hpp"
9 #include "rclcpp_lifecycle/lifecycle_node.hpp"
10 #include "std_msgs/msg/string.hpp"
11
12 namespace my_lifecycle_cpp
13 {
14
15     using CallbackReturn =
16         rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
17             CallbackReturn;
18
19     class LifecycleListener : public rclcpp_lifecycle::LifecycleNode
20     {
21     public:
22         explicit LifecycleListener(
23             const rclcpp::NodeOptions & options = rclcpp::NodeOptions());
24
25         CallbackReturn on_configure (const rclcpp_lifecycle::State &)
26             override;
27         CallbackReturn on_activate  (const rclcpp_lifecycle::State &)
28             override;
29         CallbackReturn on_deactivate(const rclcpp_lifecycle::State &)
30             override;
31         CallbackReturn on_cleanup   (const rclcpp_lifecycle::State &)
32             override;
33         CallbackReturn on_shutdown  (const rclcpp_lifecycle::State &)
34             override;
35     };
36 }
37 #endif

```

```

29
30 private:
31     void chatter_callback(const std_msgs::msg::String & msg);
32     rclcpp::Subscription<std_msgs::msg::String>::SharedPtr sub_;
33 };
34
35 } // namespace my_lifecycle_cpp
36 #endif // MY_LIFECYCLE_CPP_LIFECYCLE_LISTENER_HPP_

```

Listing 12: include/my_lifecycle_cpp/lifecycle_listener.hpp

4.7 C++ – Lifecycle Listener Implementation

```

1 // Copyright 2024 Developer
2 // SPDX-License-Identifier: Apache-2.0
3 #include "my_lifecycle_cpp/lifecycle_listener.hpp"
4
5 #include <memory>
6 #include <functional>
7
8 namespace my_lifecycle_cpp
9 {
10
11 LifecycleListener::LifecycleListener(const rclcpp::NodeOptions &
12     opts)
13 : rclcpp_lifecycle::LifecycleNode("lifecycle_listener", opts)
14 {
15     RCLCPP_INFO(get_logger(), "LifecycleListener created (
16         Unconfigured)");
17 }
18
19 CallbackReturn
20 LifecycleListener::on_configure(const rclcpp_lifecycle::State &
21     state)
22 {
23     RCLCPP_INFO(get_logger(), "Configuring: %s", state.label().c_str(
24         ));
25     sub_ = create_subscription<std_msgs::msg::String>(
26         "lifecycle_chatter", 10,
27         std::bind(&LifecycleListener::chatter_callback, this,
28             std::placeholders::_1));
29     return CallbackReturn::SUCCESS;
30 }
31
32 CallbackReturn
33 LifecycleListener::on_activate(const rclcpp_lifecycle::State &
34     state)
35 {
36     RCLCPP_INFO(get_logger(), "Activating: %s", state.label().c_str(
37         ));
38     return LifecycleNode::on_activate(state);
39 }

```

```

34
35 CallbackReturn
36 LifecycleListener::on_deactivate(const rclcpp_lifecycle::State &
   state)
37 {
38     RCLCPP_INFO(get_logger(), "Deactivating: %s", state.label().c_str
   ());
39     return LifecycleNode::on_deactivate(state);
40 }
41
42 CallbackReturn
43 LifecycleListener::on_cleanup(const rclcpp_lifecycle::State & state
   )
44 {
45     RCLCPP_INFO(get_logger(), "Cleaning up: %s", state.label().c_str
   ());
46     sub_.reset();
47     return CallbackReturn::SUCCESS;
48 }
49
50 CallbackReturn
51 LifecycleListener::on_shutdown(const rclcpp_lifecycle::State &
   state)
52 {
53     RCLCPP_INFO(get_logger(), "Shutting down: %s", state.label().
   c_str());
54     sub_.reset();
55     return CallbackReturn::SUCCESS;
56 }
57
58 void LifecycleListener::chatter_callback(const std_msgs::msg::
   String & msg)
59 {
60     RCLCPP_INFO(get_logger(), "Received: '%s'", msg.data.c_str());
61 }
62
63 } // namespace my_lifecycle_cpp
64
65 int main(int argc, char ** argv)
66 {
67     rclcpp::init(argc, argv);
68     auto node = std::make_shared<my_lifecycle_cpp::LifecycleListener
   >();
69     rclcpp::spin(node->get_node_base_interface());
70     rclcpp::shutdown();
71     return 0;
72 }

```

Listing 13: src/lifecycle_listener.cpp

4.8 C++ – Lifecycle Manager

```

1 // Copyright 2024 Developer
2 // SPDX-License-Identifier: Apache-2.0
3 /**
4  * @brief Simple lifecycle manager that orchestrates another
5  *       lifecycle node.
6  *
7  * Calls /<target>/change_state and /<target>/get_state services.
8  */
9 #include <chrono>
10 #include <memory>
11 #include <string>
12 #include <thread>
13
14 #include "rclcpp/rclcpp.hpp"
15 #include "lifecycle_msgs/srv/change_state.hpp"
16 #include "lifecycle_msgs/srv/get_state.hpp"
17 #include "lifecycle_msgs/msg/transition.hpp"
18
19 using namespace std::chrono_literals;
20 using ChangeState = lifecycle_msgs::srv::ChangeState;
21 using GetState    = lifecycle_msgs::srv::GetState;
22
23 class LifecycleManager : public rclcpp::Node
24 {
25 public:
26     explicit LifecycleManager(const std::string & target = "
27         lifecycle_talker")
28         : Node("lifecycle_manager"), target_(target)
29         {
30             change_state_client_ = create_client<ChangeState>(
31                 "/" + target + "/change_state");
32             get_state_client_ = create_client<GetState>(
33                 "/" + target + "/get_state");
34             RCLCPP_INFO(get_logger(),
35                 "LifecycleManager ready to manage: %s", target_.c_str());
36         }
37
38     // -- get_state
39     -----
40     std::string get_state()
41     {
42         if (!get_state_client_>wait_for_service(5s)) {
43             RCLCPP_ERROR(get_logger(), "get_state service unavailable");
44             return "unknown";
45         }
46         auto req = std::make_shared<GetState::Request>();
47         auto future = get_state_client_>async_send_request(req);
48         if (rclcpp::spin_until_future_complete(
49             get_node_base_interface(), future, 5s) ==
50             rclcpp::FutureReturnCode::SUCCESS)
51         {

```



```

49     return future.get()->current_state.label;
50 }
51 return "unknown";
52 }
53
54 // -- change_state
55 -----
56 bool change_state(uint8_t transition_id, const std::string &
57     label)
58 {
59     if (!change_state_client_->wait_for_service(5s)) {
60         RCLCPP_ERROR(get_logger(), "change_state service unavailable"
61             );
62         return false;
63     }
64
65     auto req = std::make_shared<ChangeState::Request>();
66     req->transition.id = transition_id;
67     req->transition.label = label;
68
69     auto future = change_state_client_->async_send_request(req);
70     if (rclcpp::spin_until_future_complete(
71         get_node_base_interface(), future, 10s) ==
72         rclcpp::FutureReturnCode::SUCCESS)
73     {
74         if (future.get()->success) {
75             RCLCPP_INFO(get_logger(),
76                 "Transition '%s' succeeded. New state: %s",
77                 label.c_str(), get_state().c_str());
78             return true;
79         }
80     }
81     RCLCPP_ERROR(get_logger(), "Transition '%s' FAILED.", label.
82         c_str());
83     return false;
84 }
85
86 // -- convenience wrappers
87 -----
88 bool configure() {
89     return change_state(
90         lifecycle_msgs::msg::Transition::TRANSITION_CONFIGURE, "
91         configure");
92 }
93
94 bool activate() {
95     return change_state(
96         lifecycle_msgs::msg::Transition::TRANSITION_ACTIVATE, "
97         activate");
98 }
99
100 bool deactivate() {
101     return change_state(

```

```

92     lifecycle_msgs::msg::Transition::TRANSITION_DEACTIVATE, "
        deactivate");
93 }
94 bool cleanup() {
95     return change_state(
96         lifecycle_msgs::msg::Transition::TRANSITION_CLEANUP, "cleanup
        ");
97 }
98 bool shutdown() {
99     return change_state(
100         lifecycle_msgs::msg::Transition::
101             TRANSITION_UNCONFIGURED_SHUTDOWN, "shutdown");
102 }
103
104 // -- demo sequence
105 -----
106 void run_demo()
107 {
108     std::this_thread::sleep_for(2s); // wait for talker to start
109     if (!configure()) return;
110     std::this_thread::sleep_for(1s);
111     if (!activate()) return;
112     std::this_thread::sleep_for(5s); // let it publish for 5 s
113     if (!deactivate()) return;
114     std::this_thread::sleep_for(1s);
115     if (!cleanup()) return;
116     RCLCPP_INFO(get_logger(), "Demo sequence complete.");
117 }
118 private:
119     std::string target_;
120     rclcpp::Client<ChangeState>::SharedPtr change_state_client_;
121     rclcpp::Client<GetState>::SharedPtr get_state_client_;
122 };
123
124 int main(int argc, char ** argv)
125 {
126     rclcpp::init(argc, argv);
127     auto manager = std::make_shared<LifecycleManager>("
        lifecycle_talker");
128     manager->run_demo();
129     rclcpp::shutdown();
130     return 0;
131 }

```

Listing 14: src/lifecycle_manager.cpp

5 Building the Packages

5.1 Workspace Layout

Both packages should live inside a colcon workspace:

```
~/ros2_ws/  
src/  
  my_lifecycle_py/    # Python package  
  my_lifecycle_cpp/   # C++ package
```

Listing 15: Workspace layout

5.2 Building

```
# Source ROS 2 underlay (replace 'humble' with your distro)  
source /opt/ros/humble/setup.bash  
  
# Navigate to workspace root  
cd ~/ros2_ws  
  
# Build all packages  
colcon build --symlink-install  
  
# Source the overlay (so your packages are found)  
source install/setup.bash
```

Listing 16: Build commands

Tip

-symlink-install links Python files directly from `src/` instead of copying them, so you can edit without rebuilding. This only applies to Python packages; C++ still needs a rebuild after source changes.

5.3 Building Individual Packages

```
# Build only the C++ package  
colcon build --packages-select my_lifecycle_cpp  
  
# Build only the Python package  
colcon build --packages-select my_lifecycle_py --symlink-install
```

Listing 17: Build a single package

6 Running and Testing

6.1 Launching the Nodes

Open three terminals, each sourced with the workspace overlay.

Terminal 1 – start the talker (Python):

```
source ~/ros2_ws/install/setup.bash  
ros2 run my_lifecycle_py lifecycle_talker
```

Terminal 2 – start the listener (Python):

```
source ~/ros2_ws/install/setup.bash
ros2 run my_lifecycle_py lifecycle_listener
```

Terminal 3 – trigger transitions manually:

6.2 Manual Transitions via CLI

The `ros2 lifecycle` CLI is the easiest way to explore the state machine.

```
# List available transitions for a node
ros2 lifecycle list /lifecycle_talker

# Get the current state
ros2 lifecycle get /lifecycle_talker

# Trigger the 'configure' transition
ros2 lifecycle set /lifecycle_talker configure

# Trigger 'activate'
ros2 lifecycle set /lifecycle_talker activate

# Watch messages being published
ros2 topic echo /lifecycle_chatter

# Deactivate -- publishing stops
ros2 lifecycle set /lifecycle_talker deactivate

# Clean up -- resources released
ros2 lifecycle set /lifecycle_talker cleanup

# Shut down
ros2 lifecycle set /lifecycle_talker shutdown
```

Listing 18: CLI transition commands

6.3 Using the Programmatic Manager

```
# In a separate terminal -- runs the full sequence automatically
ros2 run my_lifecycle_py lifecycle_manager

# Or the C++ version
ros2 run my_lifecycle_cpp lifecycle_manager
```

Listing 19: Run the lifecycle manager

6.4 Monitoring State via Topic

Lifecycle nodes automatically publish their current state on `/<node_name>/transition_event` as a `lifecycle_msgs/msg/TransitionEvent` message.

```
ros2 topic echo /lifecycle_talker/transition_event
```

Listing 20: Monitor state transitions

Sample output:

```
---
timestamp: 1711000000000000000
transition:
  id: 1
  label: configure
start_state:
  id: 1
  label: unconfigured
goal_state:
  id: 2
  label: inactive
---
```

7 Advanced Topics

7.1 Using Parameters Inside Lifecycle Nodes

Parameters are typically declared in `on_configure` so they are available for resource allocation:

```
1 def on_configure(self, state: State) -> TransitionCallbackReturn:
2     # Declare parameters with defaults
3     self.declare_parameter('publish_rate', 1.0)
4     self.declare_parameter('topic_name', 'lifecycle_chatter')
5     self.declare_parameter('queue_size', 10)
6
7     # Read them
8     rate = self.get_parameter('publish_rate').value
9     topic = self.get_parameter('topic_name').value
10    qsize = self.get_parameter('queue_size').value
11
12    # Use the parameters
13    self._pub = self.create_lifecycle_publisher(String, topic,
14        qsize)
15    self._timer_period = 1.0 / rate
16
17    self.get_logger().info(
18        f'Configured: topic={topic}, rate={rate} Hz, queue={qsize}'
19    )
20    return TransitionCallbackReturn.SUCCESS
```

Listing 21: Declaring parameters in `on_configure` (Python)

```
1 CallbackReturn LifecycleTalker::on_configure(
2     const rclcpp_lifecycle::State & state)
```

```

3 {
4     // Declare with defaults
5     declare_parameter("publish_rate", 1.0);
6     declare_parameter("topic_name", std::string("lifecycle_chatter"))
7     ;
8     declare_parameter("queue_size", 10);
9
10    // Read
11    auto rate = get_parameter("publish_rate").as_double();
12    auto topic = get_parameter("topic_name").as_string();
13    auto qsize = static_cast<size_t>(
14        get_parameter("queue_size").as_int());
15
16    // Use
17    pub_ = create_lifecycle_publisher<std_msgs::msg::String>(topic,
18        qsize);
19    timer_period_ = std::chrono::duration<double>(1.0 / rate);
20
21    RCLCPP_INFO(get_logger(),
22        "Configured: topic=%s, rate=%.1f Hz, queue=%zu",
23        topic.c_str(), rate, qsize);
24    return CallbackReturn::SUCCESS;
25 }

```

Listing 22: Declaring parameters in on_configure (C++)

7.2 Error Handling and Recovery

```

1 def on_configure(self, state: State) -> TransitionCallbackReturn:
2     try:
3         # Attempt to open hardware device
4         self._device = open_hardware('/dev/sensor0')
5         self._pub = self.create_lifecycle_publisher(
6             SensorData, 'sensor_data', 10)
7         return TransitionCallbackReturn.SUCCESS
8
9     except DeviceNotFoundError as e:
10        # FAILURE: node returns to previous primary state (
11            Unconfigured)
12        self.get_logger().error(f'Device not found: {e}. Retryable.
13            ')
14        return TransitionCallbackReturn.FAILURE
15
16    except Exception as e:
17        # ERROR: node calls on_error(); if that also fails ->
18            Finalized
19        self.get_logger().fatal(f'Unexpected error: {e}')
20        return TransitionCallbackReturn.ERROR

```

Listing 23: Returning FAILURE vs ERROR (Python)

7.3 Using a Launch File with Lifecycle Nodes

```

1  # Copyright 2024 Developer
2  # SPDX-License-Identifier: Apache-2.0
3  from launch import LaunchDescription
4  from launch_ros.actions import LifecycleNode
5  from launch_ros.actions import Node
6  from launch.actions import EmitEvent, RegisterEventHandler
7  from launch_ros.events.lifecycle import ChangeState
8  from launch_ros.event_handlers import OnStateTransition
9  from lifecycle_msgs.msg import Transition
10
11
12 def generate_launch_description():
13
14     # Declare the talker as a LifecycleNode action
15     talker = LifecycleNode(
16         package='my_lifecycle_py',
17         executable='lifecycle_talker',
18         name='lifecycle_talker',
19         namespace='',
20         output='screen',
21     )
22
23     # Declare the listener as a LifecycleNode action
24     listener = LifecycleNode(
25         package='my_lifecycle_py',
26         executable='lifecycle_listener',
27         name='lifecycle_listener',
28         namespace='',
29         output='screen',
30     )
31
32     # Automatically configure the talker when it reaches '
33     #   unconfigured'
34     configure_talker = EmitEvent(
35         event=ChangeState(
36             lifecycle_node_matcher=lambda _: True, # match all
37             transition_id=Transition.TRANSITION_CONFIGURE,
38         )
39     )
40
41     # When talker reaches 'inactive', activate it
42     activate_on_inactive = RegisterEventHandler(
43         OnStateTransition(
44             target_lifecycle_node=talker,
45             goal_state='inactive',
46             entities=[
47                 EmitEvent(event=ChangeState(
48                     lifecycle_node_matcher=lambda node: node ==

```

```

49         ))
50     ],
51 )
52 )
53
54     return LaunchDescription([
55         talker,
56         listener,
57         configure_talker,
58         activate_on_inactive,
59     ])

```

Listing 24: launch/lifecycle_demo.launch.py

```
ros2 launch my_lifecycle_py lifecycle_demo.launch.py
```

Listing 25: Running the launch file

7.4 Introspecting Lifecycle Services

Every lifecycle node exposes the following standard services:

Service	Type
/<name>/get_state	lifecycle_msgs/srv/GetState
/<name>/change_state	lifecycle_msgs/srv/ChangeState
/<name>/get_available_states	lifecycle_msgs/srv/GetAvailableStates
/<name>/get_available_transitions	lifecycle_msgs/srv/GetAvailableTransitions
/<name>/get_transition_graph	lifecycle_msgs/srv/GetAvailableTransitions

```

# Get current state
ros2 service call /lifecycle_talker/get_state \
  lifecycle_msgs/srv/GetState

# List available transitions from current state
ros2 service call /lifecycle_talker/get_available_transitions \
  lifecycle_msgs/srv/GetAvailableTransitions

# Trigger configure via service call
ros2 service call /lifecycle_talker/change_state \
  lifecycle_msgs/srv/ChangeState \
  "{transition: {id: 1, label: configure}}"

```

Listing 26: Calling lifecycle services directly

8 Best Practices

1. **Allocate in on_configure, start in on_activate.** Create publishers, subscribers, and service clients during configuration. Start timers and begin processing only on activation.

2. **Release symmetrically.** If you create something in `on_configure`, release it in `on_cleanup` (not `on_shutdown` only). Mirror every allocation with a corresponding teardown.
3. **Always call the parent implementation** for `on_activate` and `on_deactivate` when using lifecycle publishers; the parent toggles the publisher's activation flag.
4. **Use lifecycle publishers** (not regular publishers) for topics that should be silent in the `Inactive` state. Lifecycle publishers refuse to publish unless activated.
5. **Distinguish FAILURE from ERROR.** Return `FAILURE` for expected, recoverable problems (hardware not ready, parameter out of range). Return `ERROR` only for truly unexpected conditions.
6. **Keep callbacks fast.** Transition callbacks block the executor. Offload long-running initialisation (e.g. loading a large ML model) to a thread and return `SUCCESS` once the thread completes.
7. **Test each transition independently.** Write unit tests for each callback; do not rely solely on integration tests.
8. **Log state transitions.** Always log at the entry of each callback. Include the state label (`state.label()`) to make logs traceable.
9. **Avoid shared mutable state between states.** If a member variable is set in `on_configure` and used in `on_activate`, document this dependency clearly.
10. **Use a lifecycle manager for multi-node systems.** The `nav2_lifecycle_manager` package provides a production-grade manager that handles dependency ordering and error recovery.

Warning

Do **not** attempt to publish messages from a lifecycle publisher while the node is in the *Inactive* state. The message will be silently dropped. Always check `pub.is_activated()` (C++) or `pub.is_activated` (Python) before calling `publish()`.

9 Quick Reference Card

	Python	C++
Base class	<code>rclpy.lifecycle.LifecycleNode</code>	<code>rclcpp_lifecycle::LifecycleNode</code>
Return type	<code>TransitionCallbackReturn</code>	<code>CallbackReturn</code>
Lifecycle publisher	<code>create_lifecycle_publisher()</code>	<code>create_lifecycle_publisher<T>()</code>
Activate publisher	Call <code>super().on_activate(state)</code>	Call <code>LifecycleNode::on_activate(state)</code>
Check activated	<code>pub.is_activated</code>	<code>pub->is_activated()</code>
Get state (CLI)	<code>ros2 lifecycle get /<node_name></code>	
Set transition (CLI)	<code>ros2 lifecycle set /<node_name> <transition></code>	
Monitor events	<code>ros2 topic echo /<node_name>/transition_event</code>	

A Installing ROS 2 and Setting Up a Development Workspace on Ubuntu

This appendix guides you through installing ROS 2 on Ubuntu 22.04 (for *Humble Hawksbill*, the current LTS) and preparing a colcon workspace.

A.1 Supported Ubuntu Versions

ROS 2 Distribution	Ubuntu	EOL
Humble Hawksbill (LTS)	22.04 (Jammy)	May 2027
Iron Irwini	22.04 / 23.04	Nov 2024
Jazzy Jalisco (LTS)	24.04 (Noble)	May 2029

The steps below use **Humble on Ubuntu 22.04**. Replace `humble` with `jazzy` (and `jammy` with `noble`) for the Jazzy release.

A.2 Step 1 – Set the Locale

```
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

A.3 Step 2 – Add the ROS 2 APT Repository

```
# Install prerequisite tools
sudo apt install -y \
  software-properties-common \
  curl \
```

```
gnupg \  
lsb-release  
  
# Add the ROS 2 GPG key  
sudo curl -sSL \  
  https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \  
  -o /usr/share/keyrings/ros-archive-keyring.gpg  
  
# Add the repository to sources list  
echo \  
  "deb [arch=$(dpkg --print-architecture) \  
    signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \  
    https://packages.ros.org/ros2/ubuntu \  
    $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | \  
  sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null  
  
sudo apt update
```

A.4 Step 3 – Install ROS 2

```
# Full desktop install (recommended -- includes RViz, demos,  
  tutorials)  
sudo apt install -y ros-humble-desktop  
  
# OR minimal base install (headless / servers)  
# sudo apt install -y ros-humble-ros-base  
  
# Developer tools (colcon, rosdep, etc.)  
sudo apt install -y \  
  ros-dev-tools \  
  python3-colcon-common-extensions \  
  python3-rosdep \  
  python3-argcomplete
```

A.5 Step 4 – Initialise rosdep

```
sudo rosdep init  
rosdep update
```

A.6 Step 5 – Source the Installation

Add the following line to your ~/.bashrc so ROS 2 is sourced in every new terminal:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

Verify the installation:

```
ros2 --version  
# Expected output: ros2 cli... version X.Y.Z
```

A.7 Step 6 – Create and Initialise a Colcon Workspace

```
# Create the workspace directory structure
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws

# Build an empty workspace to create install/ and build/
  directories
colcon build

# Source the workspace overlay
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

A.8 Step 7 – Clone or Create Your Packages

```
# Enter the source directory
cd ~/ros2_ws/src

# Option A: clone an existing package
git clone https://github.com/ros2/demos.git

# Option B: create a new Python package from scratch
ros2 pkg create --build-type ament_python \
  --dependencies rclpy lifecycle_msgs std_msgs \
  my_lifecycle_py

# Option C: create a new C++ package from scratch
ros2 pkg create --build-type ament_cmake \
  --dependencies rclcpp rclcpp_lifecycle lifecycle_msgs std_msgs \
  my_lifecycle_cpp
```

A.9 Step 8 – Install Package Dependencies

```
# From workspace root
cd ~/ros2_ws

# Install all dependencies listed in package.xml files
rosdep install --from-paths src --ignore-src -r -y
```

A.10 Step 9 – Build and Test

```
# Full build
colcon build --symlink-install

# Build a specific package
colcon build --packages-select my_lifecycle_cpp

# Run tests
colcon test
colcon test-result --verbose
```

A.11 Step 10 – Useful Development Tools

A.11.1 colcon Cheatsheet

```
# Build with verbose output
colcon build --event-handlers console_direct+

# Build in parallel (N jobs)
colcon build --parallel-workers 4

# Build only packages that have changed
colcon build --packages-above-and-dependencies my_lifecycle_cpp

# List all packages in workspace
colcon list

# Clean build artefacts (keep install/)
rm -rf build/ log/
```

A.11.2 ROS 2 Environment Utilities

```
# List all running nodes
ros2 node list

# Inspect a node's interfaces
ros2 node info /lifecycle_talker

# List all topics
ros2 topic list

# List all services
ros2 service list

# List all lifecycle nodes
ros2 lifecycle nodes

# Echo a topic with a rate limit
ros2 topic echo /lifecycle_chatter --max-count 5
```

A.11.3 Installing Extra Packages

```
# Lifecycle messages (included in desktop; explicit install for
base)
sudo apt install -y ros-humble-lifecycle-msgs

# nav2 lifecycle manager (production-grade orchestration)
sudo apt install -y ros-humble-nav2-lifecycle-manager

# rqt plugins for graphical introspection
sudo apt install -y ros-humble-rqt-graph ros-humble-rqt-topic
```

A.11.4 Optional: Install Visual Studio Code with ROS Extension

```
# Install VS Code (if not already present)
sudo snap install code --classic

# Inside VS Code, install these extensions:
# - "ROS" by Microsoft (ms-iot.uscode-ros)
# - "C/C++" by Microsoft (ms-vscode.cpptools)
# - "Python" by Microsoft (ms-python.python)
```

A.12 Summary: Full Install Script

```
#!/usr/bin/env bash
set -e

# 1. Locale
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

# 2. Repository
sudo apt install -y software-properties-common curl gnupg lsb-
release
sudo curl -sSL \
  https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  https://packages.ros.org/ros2/ubuntu \
  $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | \
  sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
sudo apt update

# 3. Install
sudo apt install -y ros-humble-desktop ros-dev-tools \
  python3-colcon-common-extensions \
  python3-rosdep python3-argcomplete

# 4. rosdep
sudo rosdep init && rosdep update

# 5. Source
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc

# 6. Workspace
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws && colcon build
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

```
echo "ROS 2 Humble installed successfully!"
ros2 --version
```

Listing 27: One-shot install script for ROS 2 Humble on Ubuntu 22.04

Note

Run this script *once* on a fresh Ubuntu 22.04 installation. Do **not** run it inside a Docker container without adjusting the **sudo** invocations, and do **not** run as root (omit sudo and adjust paths as needed).

B Troubleshooting

Problem	Solution
ros2: command not found	Run <code>source /opt/ros/humble/setup.bash</code> or add it to <code>~/.bashrc</code> .
Node not found after build	Source the workspace overlay: <code>source ~/ros2_ws/install/setup.bash</code> .
Messages not received	Check that both talker and listener are in <i>Active</i> state. Use <code>ros2 lifecycle get</code> to inspect.
<code>change_state</code> service returns failure	The transition may not be valid from the current state. Use <code>ros2 lifecycle list</code> to see available transitions.
Publisher drops messages silently	The lifecycle publisher is inactive. Ensure you called <code>super().on_activate(state)</code> (Python) or <code>LifecycleNode::on_activate(state)</code> (C++).
Colcon build fails with missing dep	Run <code>rosdep install -from-paths src -ignore-src -r -y</code> from the workspace root.
Linking error in C++	Add <code>rclcpp_lifecycle</code> to both <code>find_package</code> and <code>ament_target_dependencies</code> in <code>CMakeLists.txt</code> .
Python <code>ImportError</code> on <code>rclpy.lifecycle</code>	Install the package: <code>sudo apt install ros-humble-rclpy</code> .